

《人工智能与机器学习》期末论文 — 技术路线

一、总体说明

1.1 论文架构

期末论文由**必做** + **选做**两部分组成：

部分	选择范围	数量	定位
必做	方向 A / 方向 B 二选一	1 个	夯实深度学习基础，体现从数据到部署的完整工程能力
选做	方向 C~H 六选一	1 个	拓展应用领域，体现框架运用和系统设计能力

两部分可独立成章，也可整合为一个大系统（如方向 A 图像分类 + 方向 F RAG 问答 = 智能校园助手）。鼓励整合，但不强求。

1.2 设计原则

所有路线设计遵循以下原则：

- 优先使用成熟开源框架，降低从零实现的成本
- 优先使用预训练模型，避免从零训练的高算力需求
- 数据获取明确可行，提供具体的数据来源和采集方案
- 每个路线标注最低硬件要求，大部分在普通笔记本上即可完成

二、必做方向（二选一）

必做方向对应课程核心模块：CNN（方向 A）与 RNN（方向 B）。两个方向均在大作业基础之上显著深化：自建数据集、多模型对比、完整 Web 系统、Grad-CAM/注意力可视化。

方向 A：迁移学习图像分类系统

难度: ★☆☆☆☆ | 对标课程: CNN + 迁移学习 | 预计工作量: 2-3 周

为什么可行

- 图像数据随手可拍，不受外部数据源限制
- PyTorch + torchvision 的迁移学习 API 成熟稳定
- Gradio 5 分钟即可生成可用的 Web 界面
- 预训练模型无需从零训练，CPU 也能完成（慢但可行）

选题示例

场景	类别数	数据采集	趣味性
校园地标识别	6-10 类	手机拍摄各教学楼/景点	★★★★
食堂菜品分类	8-15 类	食堂打饭后拍照	★★★★★
垃圾智能分类	4-6 类	收集常见垃圾拍照	★★★★
植物叶片识别	5-10 类	校园内拍摄不同植物	★★★
手写体字符识别	10-36 类	同学手写采集	★★★

分步实施计划

阶段	任务	预计时间	产出
数据采集	确定分类体系 → 手机拍摄/网络爬取每类 ≥150 张 → 按类别分文件夹存放	3-5 天	整理好的图像数据集
数据预处理	划分训练/验证/测试集 (7:2:1) → 编写 DataLoader → 配置数据增强 (随机翻转、颜色抖动、随机裁切)	1-2 天	可运行的数据加载代码
基线模型	使用 ResNet-18 预训练模型 → 替换全连接层 → 冻结骨干 → 仅训练分类头	2-3 天	第一个可运行的模型
对比实验	解冻部分层微调 vs 全微调; ResNet-18 vs MobileNetV3 vs EfficientNet-B0	3-5 天	3+ 组实验数据
可解释性	实现 Grad-CAM 热力图 → 选取典型正确/错误样本分析	1-2 天	可视化分析图
系统部署	Gradio 构建 Web UI → 上传图片 → 实时分类 + 置信度 + 热力图	1-2 天	可演示的 Web 系统
论文撰写	按章节模板撰写	贯穿全程	期末论文

数据规模建议

- 最少: 5 类 × 150 张 = 750 张 (偏少, 勉强可用)
- 推荐: 8 类 × 200 张 = 1600 张 (充足, 有余量做数据增强对比实验)
- 增强后等效规模: 原始 × 5-10 倍 (通过数据增强)

最低硬件

- 训练: 4GB 显存 (或 Colab 免费 T4 GPU) , 纯 CPU 也可完成 (约 2-5 倍时间)
- 推理: CPU 即可

论文亮点建议

- 1. 数据采集过程的记录（拍摄条件、标注规范、数据分布统计）
- 2. 至少对比 3 种预训练模型，分析精度-速度-参数量三维权衡
- 3. Grad-CAM 分析模型"为什么分类正确/错误"
- 4. 系统截图 + 实际部署体验说明

方向 B：文本情感与观点挖掘系统

难度: ★★☆☆☆ | 对标课程: RNN + 注意力机制 | 预计工作量: 2-3 周

为什么可行

- 评论文本数据可从豆瓣/美团/课程平台爬取，也可设计问卷收集
- LSTM/GRU 模型参数量小（百万级），普通电脑即可训练
- Streamlit 构建数据仪表板比 Gradio 更灵活，适合展示分析过程
- 中文数据用 jieba 分词，生态成熟

选题示例

场景	数据来源	标注方式	趣味性
课程评价情感分析	设计问卷收集对课程/教师评价	评分1-5 + 文本	★★★★
电影/美食评论挖掘	爬取豆瓣/美团评论	利用星级作为标签	★★★★★
微博/知乎舆情分析	爬取指定话题评论	人工标注正/负/中性	★★★★
App 应用商店评价	爬取 App Store/应用宝评论	星级+文本	★★★

分步实施计划

阶段	任务	预计时间	产出
数据构建	确定目标网站 → 编写爬虫/设计问卷 → 收集 ≥3000 条 → 数据清洗（去重、去噪、长度过滤）→ 标注	3-5 天	清洗标注后的文本数据集
文本预处理	jieba 分词 → 构建词表 → 文本转索引序列 → 截断/填充 → 构建 DataLoader	1-2 天	可训练的数据管道
基线模型	实现 LSTM 情感分类模型（Embedding → LSTM → FC → Softmax）	2-3 天	基线模型
对比实验	RNN vs LSTM vs GRU vs BiLSTM vs TextCNN；随机词向量 vs 预训练词向量	4-6 天	5+ 组实验数据
注意力可视化	可加入 Bahdanau 注意力 → 可视化重要词汇	1-2 天	注意力权重图
系统部署	Streamlit 仪表板：输入文本 → 情感预测 + 置信度 + 关键词高亮	1-2 天	可演示的 Web 系统

阶段	任务	预计时间	产出
论文撰写	按章节模板撰写	贯穿全程	期末论文

数据规模建议

- 最少: 3000 条 (每类 ≥1000 条)
- 推荐: 5000-8000 条 (有足够样本做细粒度分析)
- 标注策略: 可先利用星级 (4-5 星=正面, 1-2 星=负面, 3 星=中性) , 再抽查人工校正

最低硬件

- 训练: CPU 即可 (LSTM 模型参数量小) , 有 GPU 更快
- 推理: CPU

论文亮点建议

1. 数据采集全流程记录 (爬虫策略、清洗规则、标注一致性检验)
2. 5 种模型横向对比矩阵 (参数量、训练时间、准确率、F1)
3. 随机词向量 vs 预训练词向量的效果差异分析
4. 典型误分类案例分析 (为什么这句话被分错?)

三、选做方向（六选一）

选做方向拓展到更丰富的应用场景。大部分选做方向以成熟开源框架为主，重点是理解框架、适配场景、构建系统，而非从零实现算法。

方向 C：图像描述自动生成（CNN + LSTM + 注意力）

难度: ★★★☆☆ | 应用领域: 辅助视觉、内容标注 | 预计工作量: 3-4 周

为什么可行

- 编码器-解码器 + 注意力是 CV+NLP 结合的经典范式，课程已覆盖 CNN、RNN 和注意力机制
- Flickr8k (8000 张图) 数据集规模小、下载方便，单 GPU 几小时即可训练
- 注意力权重可视化直观酷炫，答辩展示效果好
- PyTorch 官方有 Image Captioning 教程可直接参考起步

推荐开源框架

工具	用途
PyTorch	编码器-解码器 + 注意力实现
torchvision	ResNet 预训练编码器
Gradio	上传图片 → 生成描述 + 注意力热力图

工具	用途
NLTK	BLEU 评分计算

分步实施计划

阶段	任务	预计时间
数据准备	下载 Flickr8k (英文) 或 AI Challenger 图像中文描述数据集	1-2 天
编码器	加载预训练 ResNet-101, 冻结权重, 提取 2048 维图像特征	1-2 天
解码器	实现 LSTM + Bahdanau 注意力解码器	3-4 天
训练	Teacher Forcing 策略训练, 监控 BLEU 分数	3-5 天
对比实验	有无注意力机制 / 不同 CNN 编码器 (ResNet vs EfficientNet) / 不同 LSTM 层数	3-4 天
可视化与系统	注意力热力图叠加原图 + Gradio 交互界面	2-3 天

简化建议

- 如果 LSTM+注意力实现困难, 可以先用 **不带注意力的编码器-解码器** 跑通基线, 再逐步添加注意力
- BLEU 计算可用 NLTK 现成实现, 不必从零写
- 推荐先从英文做起 (Flickr8k), 中文描述需要额外处理分词

最低硬件

- 训练: 4GB 显存 (Flickr8k 较小, ResNet-101 特征可预提取存到磁盘)
- 推理: CPU

方向 D: 目标检测应用系统 (YOLOv8)

难度: ★★★☆☆ | 应用领域: 安防、交通、工业 | 预计工作量: 3-4 周

为什么可行

- Ultralytics YOLOv8 是当前最易用的目标检测框架, Python API 极其友好:
`model.train(data="xxx.yaml", epochs=50)` 即可开始训练
- 数据标注工具成熟 (LabelImg、Label Studio), 上手快
- 检测结果可视化直观 (方框 + 标签 + 置信度), 成果展示效果好
- 可使用 YOLOv8 预训练权重微调, 大幅降低训练数据需求 (200-500 张标注图即可)

选题示例

场景	检测目标	数据采集	应用价值
教室人数统计	学生（人头）	教室拍照	★★★★
安全帽佩戴检测	人 + 安全帽	工地/网络图片	★★★★
车牌检测	车牌区域	停车场拍照	★★★
餐桌菜品检测	各类菜品	食堂拍照	★★★★★
交通标志检测	各类标志牌	街拍/网络图片	★★★

推荐开源框架

工具	用途
Ultralytics YOLOv8	目标检测核心框架
Labellmg / Label Studio	图像标注工具
Gradio	Web 交互界面
OpenCV	视频流读取与实时推理

分步实施计划

阶段	任务	预计时间
数据采集与标注	拍摄/收集场景图片 300-500 张 → Labellmg 标注边界框 → 导出 YOLO 格式	4-6 天
模型训练	加载 YOLOv8n/s 预训练权重 → 配置文件 → 训练 50-100 epochs	2-3 天
模型评估	mAP@50、mAP@50-95、混淆矩阵、PR 曲线	1-2 天
对比实验	YOLOv8n vs YOLOv8s / 不同数据增强强度 / 不同训练 Epoch 数	2-3 天
误检分析	分析典型 False Positive 和 False Negative，提出改进方案	1-2 天
系统部署	Gradio 上传图片/视频 → 检测框叠加 → 结果展示 + 统计计数	2-3 天

数据标注建议

- 标注量: 最少 200 张，推荐 500 张（使用预训练权重微调，数据量要求不高）
- 标注工具推荐 Labellmg（最简单）或 Label Studio（功能更丰富）
- 每张图片标注完成后导出 YOLO txt 格式
- 可用 AI 预标注 + 人工校正加速标注（YOLO 先用预训练权重跑一遍，人工修正标错的框）

最低硬件

- 训练: 4GB 显存 (YOLOv8n, batch size=8) , 推荐 8GB
- 替代方案: Google Colab 免费 T4 (16GB 显存)
- 推理: CPU 可运行 YOLOv8n

方向 E：深度学习推荐系统

难度: ★★★☆☆ | 应用领域: 电商、内容平台、教育 | 预计工作量: 3-4 周

为什么可行

- MovieLens、Amazon Reviews 等公开数据集可直接使用，无需自己爬数据
- 矩阵分解 (传统方法) → 神经协同过滤 (深度方法) 的演进路线清晰，对比实验层次分明
- 模型结构简单 (Embedding + MLP) , 训练快速
- Surprise 库封装了传统方法，一行代码出基线

选题示例

场景	数据集	数据规模
电影推荐	MovieLens 1M/10M/25M	100万~2500万条评分
图书推荐	Book-Crossing	100万条评分
课程推荐	模拟/爬取课程选择数据	自建
音乐推荐	Last.fm 数据集	收听记录

推荐开源框架

工具	用途
PyTorch	神经协同过滤 (NCF) 模型实现
Surprise	传统推荐算法基线 (SVD、KNN)
Streamlit	推荐结果展示仪表板
pandas	数据探索与交互矩阵构建

分步实施计划

阶段	任务	预计时间
数据探索	加载 MovieLens 数据集 → 分析评分分布、稀疏度、用户活跃度	1-2 天
传统基线	Surprise 实现 User-CF、Item-CF、SVD → 记录 RMSE/MAE	2-3 天
深度模型	实现 NCF: 用户 Embedding + 物品 Embedding → 拼接 → MLP → 评分预测	3-4 天

阶段	任务	预计时间
对比实验	NCF vs SVD / 不同 Embedding 维度 (8/16/32/64) / 不同 MLP 层数	3-4 天
冷启动分析	分析新用户/新物品的推荐质量，对比不同方法在冷启动下的表现	2-3 天
系统部署	Streamlit: 输入用户 ID → 展示 Top-K 推荐结果 + 预测评分	1-2 天

简化建议

- 从 MovieLens 100K (10 万条评分) 开始，数据量小、加载快，调通后换到 1M (100 万条)
- Surprise 库提供了开箱即用的 SVD 和 KNN，直接调用即可得到基线
- NCF 的模型结构较简单 (Embedding + 几层 FC)，不需要复杂的网络结构

最低硬件

- 训练: CPU 即可 (Embedding + MLP 计算量小)，MovieLens 100K 在笔记本上几分钟训练完成
- 推理: CPU

方向 F：RAG 智能知识库问答系统

难度: ★★★★★ | 应用领域: 教育、企业知识管理、客服 | 预计工作量: 3-5 周

为什么可行 (重点推荐)

- 不需要 GPU 训练：全部使用现成的嵌入模型和对话模型
- Ollama 一键部署开源大模型 (Qwen2.5/DeepSeek)，16GB 内存即可运行 7B 量化模型
- LangChain 提供了完整的 RAG 组件链 (文档加载 → 分割 → 向量化 → 检索 → 生成)
- 知识库内容可控 (课程笔记、学院制度、技术文档等)，数据获取容易
- RAG 是 2024-2026 年企业 AI 应用最主流的模式，面试高频考点

选题示例

场景	知识库内容	实用价值
课程 AI 助教	课程 PPT、教材笔记、常见 Q&A	★★★★★
校园办事指南	学院/学校规章制度、办事流程	★★★★
技术文档问答	某开源框架文档 (如 PyTorch/Vue 文档)	★★★★
学院规章制度问答	学生手册、培养方案、考试规定	★★★

推荐开源框架

工具	用途	选择建议
LangChain	RAG 应用编排	★ 推荐，社区最活跃

工具	用途	选择建议
Ollama	本地大模型部署	运行 Qwen2.5-7B / DeepSeek-R1-8B
ChromaDB	向量数据库	Python 原生，零配置启动
BGE / M3E	中文文本嵌入模型	通过 HuggingFace 加载
Chainlit	对话式 Web UI	专为 LLM 对话应用设计

技术架构（简化版）



分步实施计划

阶段	任务	预计时间
环境搭建	安装 Ollama → 拉取 Qwen2.5-7B → 安装 LangChain + ChromaDB → 验证可运行	1-2 天
知识库构建	收集文档（PDF/网页/MD）→ 文本清洗 → 文本分块（Chunk）→ BGE 嵌入 → 存入 ChromaDB	3-5 天
检索链路	LangChain 实现：用户问题 → 嵌入 → ChromaDB 检索 → 拼接 Prompt → Ollama 生成	2-3 天
对比实验	不同 Chunk Size（256/512/1024）对比 / 不同 Top-K（3/5/10）对比 / 不同嵌入模型对比	3-4 天
对话功能	实现多轮对话（对话历史管理 + 上下文窗口）	2-3 天
系统部署	Chainlit 对话界面 → 知识库管理（上传/更新文档）	2-3 天
评测	构建 30+ 测试问题，从准确性、完整性评估回答质量	2-3 天

关键超参数调优指南

参数	建议范围	调优方向
Chunk Size	256 / 512 / 1024	越小越精确但丢失上下文，越大上下文越完整但检索精度下降
Chunk Overlap	Chunk Size 的 10%-20%	保证语义不被切断
Top-K	3 / 5 / 10	K 太小信息不足，K 太大可能引入噪音

参数	建议范围	调优方向
嵌入模型	BGE-large / M3E-large	对比不同模型在同一知识库上的检索精度

最低硬件

- 本地模型方案: 16GB 内存 (Ollama 4-bit 量化 7B 模型, 推理速度 5-15 token/s)
- 云端 API 方案: 仅需 CPU 和网络 (使用 DeepSeek API / 通义千问 API, 无硬件要求)

常见踩坑提醒

- Ollama 下载慢: 设置国内镜像源或使用 HF Mirror
- PDF 解析乱码: 用 PyMuPDF 或 pdfplumber 替代 PyPDF2
- 检索结果不相关: 调整 Chunk Size 和 Overlap, 检查嵌入模型是否适合中文
- 模型回答太慢: 换更小的量化版本 (如 Qwen2.5-1.5B 用于测试, 7B 用于正式)

方向 G：大模型驾驭工程 — 多 Agent 协作系统

难度: ★★★★★ | 应用领域: 自动化办公、AI 工作流 | 预计工作量: 3-5 周

为什么可行

- 不需要 GPU 训练: 用 CrewAI 编排多个 Agent, 每个 Agent 背后是 API 或 Ollama 推理
- CrewAI 专为降低多 Agent 开发门槛而设计, API 简洁, 文档友好
- 选题灵活、趣味性强, 学生可以发挥创意
- 重点是系统架构设计和 Prompt 工程, 这正是软件工程专业的核心能力

选题示例

场景	Agent 角色设计	趣味性
AI 内容工作室	策划 Agent + 撰稿 Agent + 审核 Agent	★★★★★
AI 旅行规划师	需求分析 Agent + 行程规划 Agent + 预算 Agent	★★★★
课程学习助手	知识讲解 Agent + 出题 Agent + 批改 Agent	★★★★
代码审查小助手	代码分析 Agent + 安全检查 Agent + 优化建议 Agent	★★★★★
智能辩论系统	正方 Agent + 反方 Agent + 裁判 Agent	★★★★★

推荐开源框架

工具	用途	选择建议
CrewAI	多 Agent 编排	★推荐, 最易上手
Ollama	本地模型运行	运行 Agent 的"大脑"
Chainlit	对话式 UI	展示多 Agent 对话过程

工具	用途	选择建议
DeepSeek API / 智谱 API	云端模型（备选）	更稳定、更强，但需付费（约 1-2 元/百万 token）

CrewAI 核心概念速览

```
from crewai import Agent, Task, Crew

# 1. 定义 Agent（角色 + 目标 + 背景故事）
planner = Agent(
    role="内容策划师",
    goal="分析用户需求，制定内容大纲",
    backstory="你是一位资深内容策划，擅长分析目标受众...",
    llm=ollama_llm, # 或使用云端 API
)

# 2. 定义 Task（任务描述 + 期望输出）
plan_task = Task(
    description="根据用户主题 {topic}，制定文章大纲...",
    expected_output="包含标题、小标题、关键词的大纲",
    agent=planner,
)

# 3. 组装 Crew（多名 Agent + 多个 Task）
crew = Crew(agents=[planner, writer, reviewer], tasks=[...])
result = crew.kickoff(inputs={"topic": "Python异步编程"})
```

分步实施计划

阶段	任务	预计时间
场景设计	确定应用场景 → 设计 Agent 角色（2-4 个）→ 定义各 Agent 的 Role/Goal/Backstory → 设计协作流程	2-3 天
单 Agent 调试	逐个实现和调试每个 Agent 的 Prompt → 确保单个 Agent 输出质量达标	2-3 天
协作串联	使用 CrewAI 编排多 Agent 顺序/并行协作 → 调试 Agent 间信息传递	3-4 天
工具集成	为 Agent 绑定工具（搜索 API、计算器、文件读写等）	2-3 天
对比实验	单 Agent vs 多 Agent / 有搜索工具 vs 无工具 / Qwen2.5 vs DeepSeek	3-4 天
系统部署	Chainlit 对话界面 → 用户输入任务 → 展示各 Agent 协作过程	2-3 天

简化建议

- 从 2 个 Agent（如策划 + 撰稿）开始，调通后扩展到 3-4 个
- 先用云端 API（DeepSeek/智谱）开发调试，速度快、效果好，最后再考虑切换 Ollama 本地模型
- 工具集成可从最简单的搜索工具（Tavily Search API，免费额度够用）开始

最低硬件

- **云端 API 方案 (推荐)** : 仅需 CPU + 网络, 无硬件门槛
- **Ollama 本地方案**: 16GB 内存 (运行 7B 量化模型, 但多 Agent 轮流推理会稍慢)

论文亮点建议

1. Agent 角色设计的思路和迭代过程
2. 不同 Prompt 策略对 Agent 输出质量的影响 (附对比案例)
3. 单 Agent vs 多 Agent 协作的任务完成质量对比
4. Agent 协作流程的架构图 (论文核心图)

方向 H：多模态语义检索系统

难度: ★★★★★ | **应用领域:** 数字资产管理、智能搜索 | **预计工作量:** 3-5 周

为什么可行

- Chinese-CLIP 提供了开箱即用的预训练模型, **只需推理不需训练**
- CLIP 将图像和文本映射到同一向量空间的原理清晰优雅
- 检索效果好、直观——输入文字就能搜到相关图片
- FAISS 提供 Python 接口, 几行代码完成向量检索

选题示例

场景	图像库	检索方式
个人相册智能搜索	手机相册 500-2000 张	文搜图 ("去年夏天在海边")
表情包语义搜索	收集表情包 300-1000 张	文搜图 ("开心的猫")
校园风景检索	校园照片 200-500 张	文搜图 + 图搜图
商品以图搜物	某品类商品图	图搜图 (上传找相似款)

推荐开源框架

工具	用途
Chinese-CLIP	阿里达摩院开源的中文图文模型 (核心)
FAISS	Meta 开源的高性能向量检索库
Gradio	图文混合检索 Web UI
HuggingFace Transformers	加载 CLIP 模型

分步实施计划

阶段	任务	预计时间
模型准备	加载 Chinese-CLIP (ViT-B/16) → 验证文本编码和图像编码功能	1-2 天
图像库构建	收集 500-2000 张图片 → CLIP 图像编码器提取特征向量 → 存入 FAISS 索引	2-3 天
文搜图	用户输入文本 → CLIP 文本编码 → FAISS 检索 Top-K → 返回图片	2-3 天
图搜图	用户上传图片 → CLIP 图像编码 → FAISS 检索 → 返回相似图片	1-2 天
对比实验	CLIP vs 颜色直方图 (传统方法) / 不同 CLIP 变体 (ViT-B vs ViT-L) / 不同 FAISS 索引类型	3-4 天
系统部署	Gradio 构建双模式检索界面 (文搜图 + 图搜图)	2-3 天

简化建议

- 图像库控制在 1000 张以内，FAISS Flat 索引即可，不需要复杂的 IVF/HNSW 索引
- 先从文搜图做起（最直观），图搜图作为扩展
- Chinese-CLIP 模型较小 (ViT-B/16 约 400MB)，下载快、推理快
- 如果 FAISS 安装困难，可先用 numpy + 余弦相似度的朴素实现，后面再升级

最低硬件

- 特征提取: CPU 可完成 (1000 张图约 5-10 分钟)，有 GPU 更快
- 检索: CPU，FAISS 毫秒级响应
- 不需要 GPU 训练

四、组合建议

4.1 必做 + 选做的搭配推荐

必做	选做	整合方式	适合人群
A (图像分类)	D (目标检测)	统一 Web 平台：分类+检测	对 CV 感兴趣
A (图像分类)	F (RAG 问答)	智能校园助手：拍照识别+知识问答	想做综合应用
A (图像分类)	H (多模态检索)	从单模态分类到跨模态检索的技术演进	想深入多模态
B (情感分析)	F (RAG 问答)	文本分析+知识问答的统一文本智能平台	对 NLP 感兴趣
B (情感分析)	G (多 Agent)	Agent 调用情感分析模型做舆情监控	想做大模型应用

必做	选做	整合方式	适合人群
B (情感分析)	E (推荐系统)	基于评论情感的推荐 (情感增强推荐)	想做算法结合

4.2 配套工具速查

功能	推荐工具	替代工具
模型训练框架	PyTorch	—
Web 界面 (CV 类)	Gradio	Streamlit
Web 界面 (NLP/对话类)	Chainlit / Streamlit	Gradio
本地大模型部署	Ollama	llama.cpp
大模型 API	DeepSeek API (便宜)	通义千问 / 智谱 GLM
向量数据库	ChromaDB (轻量)	FAISS / Milvus Lite
数据标注 (CV)	LabelImg / Label Studio	CVAT
中文分词	jieba	pkuseg
代码管理	Git + GitHub	—

五、其他建议

5.1 论文写作建议

- **不要写成实验报告。**论文要有"故事线": 为什么选这个题目 → 面临什么挑战 → 用什么方法解决 → 效果如何 → 有什么不足
- **对比实验是论文分析的核心。**不要只给数字表格, 要分析为什么 A 比 B 好, 什么场景下 B 更合适
- **系统和模型不是一回事。**论文要体现你不仅训练了模型, 还把它做成了一个可用的系统
- **记录过程而非只报告结果。**你试过但失败的方法、你遇到并解决的坑, 这些内容比"准确率 95%"更有价值
- **框架应用说明集中写。**在"系统设计与方法"一章单独用一个小节说明你在框架上做了哪些定制化工作

5.2 实验设计建议

- 每组对比实验**只改变一个变量** (控制变量法), 否则无法归因
- 实验至少跑 2-3 次报告均值和标准差 (深度学习训练有随机性)
- 固定随机种子 (`torch.manual_seed(42)`) 保证可复现
- 训练曲线 (loss/accuracy 随 epoch 变化) 是重要的实验记录, 用 tensorboard 或 matplotlib 记录
- 除了模型指标, 也记录训练时间、推理速度、模型大小等工程指标

5.3 常见问题提醒

问题	怎么避免
选题太大做不完	缩小范围，先做核心功能，有时间再扩展
花了太多时间调参	先跑通基线，超参数调优放在实验对比阶段
论文像技术文档	参考《动手学深度学习》的写作风格——有原理、有代码、有分析
数据集质量差导致模型效果不佳	论文中如实记录数据问题，分析数据质量对模型的影响本身就是有价值的讨论
只报告了最终结果	记录每次实验的参数配置和结果，方便写论文时回溯
代码一次写完没做版本管理	从一开始就用 Git，每次有意义的变化 commit 一次

5.4 时间管理建议

阶段	建议时间	关键提醒
选题 + 技术调研	第 1 周	尽快确定方向，不要反复更换
数据采集与处理	第 1-2 周	最早开始、最耗时的环节
基线模型跑通	第 2 周	不求效果最好，先跑通完整流程
对比实验	第 2-3 周	在基线基础上系统性地开展实验
系统界面开发	第 3 周	与实验并行推进
论文初稿	第 3-4 周	一边做实验一边写，不要攒到最后
修改完善 + 录视频	第 4 周	预留足够时间打磨

5.5 关于 AI 工具的使用

- **允许:** 用 AI 辅助理解概念、调试 error、生成代码片段（你理解后修改使用）、润色论文语言
- **不允许:** 整段/整篇 AI 生成不经修改直接使用、把题目输入 AI 让它"生成完整代码"
- **判断标准:** 你是否能向别人解释你的每一行代码、每一个设计决策？如果不能，那就需要重新审视

各方向的具体选题可与任课教师讨论确认。鼓励在给定路线基础上提出自己的创新想法。